
A10\code\A10.py

```
import numpy as np
import matplotlib.pyplot as plt
import time
from utils import *

def two_loop_lbfgs(q, s, y, B0k):
    """
    Two-Loop L-BFGS Recursion

    Input:
    q: Vektor (alte Suchrichtung),
    s: list(nd.array(n,)) (Liste der letzten m Vektoren sk-1, ..., sk-m)
    y: list(nd.array(n,)) (Liste der letzten m Vektoren yk-1, ..., yk-m)
    B0k: Matrix (Schätzung für die inverse Hesse-Matrix zum Zeitpunkt k-m)

    Output:
    p: Vektor (neue Suchrichtung)
    """
    p = None
    # TODO: Aufgabenteil 1. Two-Loop L-BFGS Recursion implementieren.
    # BEGIN SOLUTION
    roh = np.zeros(len(s))
    alpha = np.zeros(len(s))
    for i in range(len(s)):
        roh[i] = 1/np.dot(y[i], s[i])
        alpha[i] = roh[i] * np.dot(s[i], q)
        q -= alpha[i] * y[i]
    p = B0k @ q
    for i in range(len(s) - 1, -1, -1):
        beta = roh[i] * np.dot(y[i], p)
        p += (alpha[i] - beta) * s[i]
    # END SOLUTION
    return p

def lbfgs(f, x0, B0, m, rho=0.5, c1=1e-3, eps=1e-10, tau=1e-10, kmax=100, plot=True, display=True):
    """
    Limited-Memory BFGS-basiertes Quasi-Newton-Verfahren zur Minimierung der Funktion f mit
    Backtracking-Liniensuche, Abbruchkriterien nach Gill-Murray-Wright und Two-Loop L-BFGS Recursion.

    Input:
    f: Funktion, wobei f(x) = (val, grad, hess),
    x0: Vektor (Startwert),
    B0: Matrix (Initiale Schätzung für die inverse Hesse-Matrix),
    m: Integer (Memory-Größe im l-BFGS-Verfahren)
```

```

    rho, c1: Skalare (Armijo Parameter),
    tau, eps, kmax: Skalare (Abbruchkriterien nach Gill-Murray-Wright),
    plot: Boolescher Wert (gibt an, ob Daten zur Ploterstellung gespeichert werden sollen),
    display: Boolescher Wert (gibt an, ob Zwischenwerte aus den Iterationen in der Konsole ausgegeben

Output:
    log: Dictionary vom Verlauf der Optimierung.
    """

# Dictionary zum Abspeichern der Ergebnisse.
log = {
    'x0': x0,                # Startwert
    'xlbfgs': None,          # Lösung des CG-Verfahrens
    'klbfgs': None,          # Anzahl benötigter Iterationen
    'x_list': [],            # Liste der Iterierten
    'val_list': [],          # Liste des Funktionswerts in den Iterierten
    'norm_grad_list': [],    # Liste der Norm des Gradienten in den Iterierten
}

# TODO: Aufgabenteil 2. L-BFGS-Verfahren implementieren.
# Hinweis. Die Listen-Methoden pop() und insert() können hilfreich seien.
# BEGIN SOLUTION
# Verwenden Quasi-Newton() aus utils.py als Basis

# Initialisierung VERAENDERT
xk = x0
valk, gradk, _ = f(xk)
k = 0
s_list = []
y_list = []

# Iteration
while True:
    if plot:
        log['x_list'].append(xk)
        log['val_list'].append(valk)
        log['norm_grad_list'].append(np.linalg.norm(gradk))

    # Bestimme Suchrichtung NEU
    if len(s_list) > 0:
        pk = -two_loop_lbfgs(gradk, s_list, y_list, B0)
    else:
        pk = -B0 @ gradk

    # Alt
    # Liniensuche
    alphak = armijo(f, p=pk, x=xk, rho=rho, c1=c1)

    if display:
        print(f'Iteration k = {k}: \talpha_k = {alphak: .4e}, '
              f'\txk = {str(np.round(xk, 4).flatten()): <30}, \tf(xk) = {np.round(valk, 4)}')

    xkplus1 = xk + alphak * pk
    valkplus1, gradkplus1, _ = f(xkplus1)

    # Test der Abbruchkriterien

```

```

    if gill_murray_wright(xk, xkplus1, valk, valkplus1, gradk, k, tau, eps, kmax):
        break

    # Anpassung von Bk
    sk = xkplus1 - xk
    yk = gradkplus1 - gradk

    # Warnung, wenn yk @ sk nicht > 0
    if sk @ yk < 1e-12:
        print(f"ACHTUNG: sk @ yk = {sk @ yk}")

    # NEU
    # Füge s^k und y^k zu den Listen hinzu
    s_list.append(sk)
    y_list.append(yk)
    # Falls bereits m Vektoren in Listen: entferne das Älteste Element
    if len(s_list) > m:
        s_list.pop(0)
        y_list.pop(0)

    # Update ALT
    xk, valk, gradk = xkplus1, valkplus1, gradkplus1
    k += 1

log['xlbfgs'] = xk
log['klbfgs'] = k
# END SOLUTION
return log

if __name__ == '__main__':
    m = 2
    x0 = np.array([.5, .5])
    tau = 1e-6
    eps = 1e-10
    kmax = 500
    rho = .5
    c1 = 1e-3

    f = lambda x: rosenbrock(*x)
    B0 = np.eye(2)

    # TODO: Aufgabenteil 3. l-BFGS-Verfahren testen
    # BEGIN SOLUTION
    log_lbfgs = lbfgs(f, x0, B0, m, rho=rho, c1=c1, tau=tau, eps=eps, kmax=kmax, plot=False, display=True)
    print(f"LÃ¶sung: {log_lbfgs['xlbfgs']}")
    print(f"Anzahl der Iterationen: {log_lbfgs['klbfgs']}")
    # END SOLUTION

    # TODO. Aufgabenteil 4. Vergleich von l-BFGS-, Quasi-Newton-, Newton- und Gradientenabstiegsverfahren
    # Hinweis: Ein Implementierungsvorschlag der "Älteren" Verfahren ist in utils.py gegeben
    # BEGIN SOLUTION
    # Erstellen x0-Werte
    x0_werte = [np.array([-1.2, 1.0]), np.array([0.0, -0.71])]

    # Bestimmen Verfahrensnamen und deren Anwendung

```

```

verfahren = [("l-BFGS (m=2)", lambda x0: lbfgs(f, x0, B0, m, rho=rho, c1=c1, tau=tau, eps=eps, kmax=kmax),
("BFGS", lambda x0: quasi_newton(f, x0, B0, rho=rho, c1=c1, tau=tau, eps=eps, kmax=kmax, plot=True),
("Newton", lambda x0: newton_erweitert(f, x0, rho=rho, c1=c1, tau=tau, eps=eps, kmax=kmax)),
("Gradientenverfahren", lambda x0: gradient_descent_erweitert(f, x0, rho=rho, c1=c1, tau=tau, eps=

```

```

# Iteration über Startwerte und Verfahren

```

```

for x0 in x0_werte:
    for name, anwendung in verfahren:
        log = anwendung(x0)
        fig = plot_iteration_rosenbrock(log, f"{name}, Startwert: {x0}")
        plt.show()
# END SOLUTION

```

```

# TODO. Aufgabenteil 4. Diskussion

```

```

# BEGIN SOLUTION

```

```

print("Es zeigt sich, dass Newton die wenigsten Iterationen benötigt, dies geht natürlich auf Kosten
"Das Gradientenverfahren benötigt immer die kmax Iteration und terminiert durch GMW 3. Der bere
"Das BFGS-Verfahren und das l-BFGS-Verfahren benötigen beide (deutlich) weniger Iterationen als
"wobei l-BFGS deutlich weniger Speicher benötigt. Dabei ist zu beobachten, dass abhängig vom S
"Verfahren fast so schnell wie Newton (nach Iterationen) konvergiert, beim anderen Punkt aber de
"Wobei selbst hier das l-BFGS-Verfahren nicht so viele Iterationen benötigt wie das BFGS beim a

```

```

# END SOLUTION

```

```

#TERMINAL OUTPUT:

```

```

Iteration k = 0: alpha_k = 3.9062e-03, xk = [0.5 0.5] , f(xk) = 6.5
Iteration k = 1: alpha_k = 6.2500e-02, xk = [0.6992 0.3047] , f(xk) = 3.4841
Iteration k = 2: alpha_k = 1.9531e-03, xk = [0.4905 0.0722] , f(xk) = 3.0948
Iteration k = 3: alpha_k = 2.4414e-04, xk = [0.6688 0.2818] , f(xk) = 2.8501
Iteration k = 4: alpha_k = 3.9062e-03, xk = [0.3675 0.051 ] , f(xk) = 1.1063
Iteration k = 5: alpha_k = 7.8125e-03, xk = [0.8902 0.7028] , f(xk) = 0.8152
Iteration k = 6: alpha_k = 3.9062e-03, xk = [0.4945 0.1958] , f(xk) = 0.4937
Iteration k = 7: alpha_k = 5.0000e-01, xk = [0.469 0.22 ] , f(xk) = 0.282
Iteration k = 8: alpha_k = 7.8125e-03, xk = [0.762 0.5493] , f(xk) = 0.1549
Iteration k = 9: alpha_k = 1.5259e-05, xk = [0.7423 0.5213] , f(xk) = 0.1543
Iteration k = 10: alpha_k = 4.8828e-04, xk = [0.6929 0.4565] , f(xk) = 0.15
ACHTUNG: sk @ yk = -0.006463909374272658
Iteration k = 11: alpha_k = 7.6294e-06, xk = [0.7652 0.5569] , f(xk) = 0.1374
ACHTUNG: sk @ yk = -0.0012649770730324417
Iteration k = 12: alpha_k = 4.7684e-07, xk = [0.7701 0.5641] , f(xk) = 0.1372
ACHTUNG: sk @ yk = -0.2579171820317603
Iteration k = 13: alpha_k = 7.8125e-03, xk = [0.7378 0.5194] , f(xk) = 0.1308
Iteration k = 14: alpha_k = 1.5259e-05, xk = [0.7679 0.5625] , f(xk) = 0.1274
Iteration k = 15: alpha_k = 9.5367e-07, xk = [0.7699 0.5654] , f(xk) = 0.1274
Iteration k = 16: alpha_k = 9.5367e-07, xk = [0.7722 0.5688] , f(xk) = 0.1274
Iteration k = 17: alpha_k = 3.8147e-06, xk = [0.7719 0.5683] , f(xk) = 0.1274

```

ACHTUNG: sk @ yk = -0.002022753534685534
 Iteration k = 18: alpha_k = 1.4901e-08, xk = [0.7816 0.5828] , f(xk) = 0.1268
 ACHTUNG: sk @ yk = -0.056779338359224774
 Iteration k = 19: alpha_k = 3.9062e-03, xk = [0.7687 0.5639] , f(xk) = 0.1262
 Iteration k = 20: alpha_k = 3.8147e-06, xk = [0.7791 0.5792] , f(xk) = 0.126
 Iteration k = 21: alpha_k = 4.7684e-07, xk = [0.781 0.5821] , f(xk) = 0.126
 ACHTUNG: sk @ yk = -9.275418097712262e-06
 Iteration k = 22: alpha_k = 5.9605e-08, xk = [0.7787 0.5786] , f(xk) = 0.126
 ACHTUNG: sk @ yk = -0.004676858707758913
 Iteration k = 23: alpha_k = 5.9605e-08, xk = [0.7874 0.5916] , f(xk) = 0.1259
 Iteration k = 24: alpha_k = 2.4414e-04, xk = [0.7793 0.5796] , f(xk) = 0.1255
 Iteration k = 25: alpha_k = 1.1921e-07, xk = [0.7804 0.5813] , f(xk) = 0.1255
 Iteration k = 26: alpha_k = 5.9605e-08, xk = [0.7854 0.5887] , f(xk) = 0.1254
 ACHTUNG: sk @ yk = -0.004904571129096147
 Iteration k = 27: alpha_k = 1.2207e-04, xk = [0.7756 0.5741] , f(xk) = 0.1254
 Iteration k = 28: alpha_k = 7.6294e-06, xk = [0.7816 0.5831] , f(xk) = 0.1253
 Iteration k = 29: alpha_k = 2.3842e-07, xk = [0.7841 0.5867] , f(xk) = 0.1253
 ACHTUNG: sk @ yk = -9.469754449588363e-05
 Iteration k = 30: alpha_k = 1.9073e-06, xk = [0.7812 0.5824] , f(xk) = 0.1253
 ACHTUNG: sk @ yk = -0.0004872523315890276
 Iteration k = 31: alpha_k = 2.9802e-08, xk = [0.7894 0.5947] , f(xk) = 0.1252
 Iteration k = 32: alpha_k = 6.1035e-05, xk = [0.7809 0.5821] , f(xk) = 0.1249
 Iteration k = 33: alpha_k = 4.7684e-07, xk = [0.7865 0.5905] , f(xk) = 0.1248
 Iteration k = 34: alpha_k = 1.1642e-10, xk = [0.7866 0.5905] , f(xk) = 0.1248
 ACHTUNG: sk @ yk = -0.00019300434507912856
 Iteration k = 35: alpha_k = 2.3842e-07, xk = [0.7858 0.5894] , f(xk) = 0.1248
 ACHTUNG: sk @ yk = -4.74616773997287e-06
 Iteration k = 36: alpha_k = 1.4901e-08, xk = [0.7867 0.5907] , f(xk) = 0.1248
 Iteration k = 37: alpha_k = 3.7253e-09, xk = [0.7865 0.5904] , f(xk) = 0.1248
 Iteration k = 38: alpha_k = 3.7253e-09, xk = [0.7863 0.5902] , f(xk) = 0.1248
 Iteration k = 39: alpha_k = 1.1921e-07, xk = [0.7864 0.5903] , f(xk) = 0.1248
 Iteration k = 40: alpha_k = 2.3283e-10, xk = [0.7837 0.5862] , f(xk) = 0.1248
 Iteration k = 41: alpha_k = 7.8125e-03, xk = [0.7876 0.592] , f(xk) = 0.1247
 Iteration k = 42: alpha_k = 7.4506e-09, xk = [0.7883 0.5931] , f(xk) = 0.1247
 Iteration k = 43: alpha_k = 9.5367e-07, xk = [0.7816 0.5831] , f(xk) = 0.1247
 Iteration k = 44: alpha_k = 1.5259e-05, xk = [0.7912 0.5976] , f(xk) = 0.1244
 Iteration k = 45: alpha_k = 1.5259e-05, xk = [0.7876 0.5922] , f(xk) = 0.1243
 Iteration k = 46: alpha_k = 5.9605e-08, xk = [0.7792 0.5797] , f(xk) = 0.1241

ACHTUNG: sk @ yk = -0.016725294812051983
 Iteration k = 47: alpha_k = 1.5625e-02, xk = [0.7907 0.5969] , f(xk) = 0.1236
 Iteration k = 48: alpha_k = 4.7684e-07, xk = [0.7426 0.5291] , f(xk) = 0.1163
 ACHTUNG: sk @ yk = -0.7298403820294439
 Iteration k = 49: alpha_k = 3.9062e-03, xk = [0.8093 0.6281] , f(xk) = 0.1087
 Iteration k = 50: alpha_k = 1.5259e-05, xk = [0.7185 0.5021] , f(xk) = 0.0995
 ACHTUNG: sk @ yk = -0.037462006997543096
 Iteration k = 51: alpha_k = 1.2207e-04, xk = [0.7859 0.5974] , f(xk) = 0.0866
 ACHTUNG: sk @ yk = -0.011846214929547738
 Iteration k = 52: alpha_k = 7.6294e-06, xk = [0.743 0.5394] , f(xk) = 0.0821
 Iteration k = 53: alpha_k = 9.7656e-04, xk = [0.7731 0.5815] , f(xk) = 0.0778
 Iteration k = 54: alpha_k = 1.5259e-05, xk = [0.7632 0.5678] , f(xk) = 0.0776
 Iteration k = 55: alpha_k = 7.6294e-06, xk = [0.7576 0.5602] , f(xk) = 0.0775
 ACHTUNG: sk @ yk = -0.0015312980044426827
 Iteration k = 56: alpha_k = 1.9531e-03, xk = [0.7708 0.5784] , f(xk) = 0.0774
 ACHTUNG: sk @ yk = -0.09914049377004533
 Iteration k = 57: alpha_k = 1.5259e-05, xk = [0.8041 0.6588] , f(xk) = 0.0534
 Iteration k = 58: alpha_k = 1.0000e+00, xk = [0.8961 0.7839] , f(xk) = 0.0473
 Iteration k = 59: alpha_k = 9.7656e-04, xk = [0.8386 0.6892] , f(xk) = 0.0458
 ACHTUNG: sk @ yk = -0.08399163898419268
 Iteration k = 60: alpha_k = 7.8125e-03, xk = [0.8715 0.7664] , f(xk) = 0.0211
 Iteration k = 61: alpha_k = 1.5625e-02, xk = [0.9351 0.8649] , f(xk) = 0.0131
 Iteration k = 62: alpha_k = 3.1250e-02, xk = [0.8998 0.811] , f(xk) = 0.0103
 Iteration k = 63: alpha_k = 1.5259e-05, xk = [0.9984 1.0047] , f(xk) = 0.0062
 ACHTUNG: sk @ yk = -0.1856276126521064
 Iteration k = 64: alpha_k = 1.2500e-01, xk = [1.0076 1.0105] , f(xk) = 0.0023
 Iteration k = 65: alpha_k = 1.5259e-05, xk = [1.004 1.0032] , f(xk) = 0.0022
 Iteration k = 66: alpha_k = 1.2207e-04, xk = [0.9769 0.9503] , f(xk) = 0.0022
 ACHTUNG: sk @ yk = -0.002574752769709643
 Iteration k = 67: alpha_k = 1.9531e-03, xk = [1.0094 1.0155] , f(xk) = 0.0013
 Iteration k = 68: alpha_k = 5.9605e-08, xk = [1.0105 1.0176] , f(xk) = 0.0013
 Iteration k = 69: alpha_k = 9.7656e-04, xk = [0.9931 0.9832] , f(xk) = 0.001
 Iteration k = 70: alpha_k = 4.8828e-04, xk = [1.0158 1.0298] , f(xk) = 0.0007
 Iteration k = 71: alpha_k = 9.7656e-04, xk = [1.0058 1.0093] , f(xk) = 0.0006
 Iteration k = 72: alpha_k = 7.8125e-03, xk = [1.0022 1.0045] , f(xk) = 0.0
 Iteration k = 73: alpha_k = 1.0000e+00, xk = [0.9984 0.9968] , f(xk) = 0.0
 ACHTUNG: sk @ yk = -1.0376350411803439e-07
 Iteration k = 74: alpha_k = 3.1250e-02, xk = [0.9998 0.9997] , f(xk) = 0.0

Iteration k = 75: $\alpha_k = 1.9531e-03$, $x_k = [0.9999 \ 0.9998]$, $f(x_k) = 0.0$
 ACHTUNG: $s_k @ y_k = -3.5726479315621006e-07$

Iteration k = 76: $\alpha_k = 6.2500e-02$, $x_k = [1.0001 \ 1.0002]$, $f(x_k) = 0.0$
 Iteration k = 77: $\alpha_k = 4.8828e-04$, $x_k = [0.9999 \ 0.9998]$, $f(x_k) = 0.0$
 Iteration k = 78: $\alpha_k = 3.9062e-03$, $x_k = [0.9999 \ 0.9997]$, $f(x_k) = 0.0$
 ACHTUNG: $s_k @ y_k = -2.8076609237676656e-09$

Iteration k = 79: $\alpha_k = 9.7656e-04$, $x_k = [0.9999 \ 0.9998]$, $f(x_k) = 0.0$
 ACHTUNG: $s_k @ y_k = -6.923605908555583e-08$

Iteration k = 80: $\alpha_k = 4.8828e-04$, $x_k = [1. \ 1.]$, $f(x_k) = 0.0$
 Iteration k = 81: $\alpha_k = 5.0000e-01$, $x_k = [1.0001 \ 1.0001]$, $f(x_k) = 0.0$
 Iteration k = 82: $\alpha_k = 1.2500e-01$, $x_k = [1.0001 \ 1.0001]$, $f(x_k) = 0.0$
 Iteration k = 83: $\alpha_k = 1.5259e-05$, $x_k = [1. \ 1.]$, $f(x_k) = 0.0$
 Iteration k = 84: $\alpha_k = 2.5000e-01$, $x_k = [1. \ 1.]$, $f(x_k) = 0.0$

Abbruch nach GMW 1 erfuehlt

Lösung: $[0.99998529 \ 0.99997254]$

Anzahl der Iterationen: 84

ACHTUNG: $s_k @ y_k = -0.12884777380750428$
 ACHTUNG: $s_k @ y_k = -0.4402041987859732$
 ACHTUNG: $s_k @ y_k = -5.5677834857590645$
 ACHTUNG: $s_k @ y_k = -0.00812933600271466$
 ACHTUNG: $s_k @ y_k = -0.0013455214651268864$
 ACHTUNG: $s_k @ y_k = -0.0352095484420171$
 ACHTUNG: $s_k @ y_k = -42.551731241489094$
 ACHTUNG: $s_k @ y_k = -0.8892706260302573$
 ACHTUNG: $s_k @ y_k = -0.0005935018825419663$
 ACHTUNG: $s_k @ y_k = -0.0007719923951601676$
 ACHTUNG: $s_k @ y_k = -0.0011613399230350763$
 ACHTUNG: $s_k @ y_k = -0.00046177096654965305$
 ACHTUNG: $s_k @ y_k = -3251.1847895544397$
 ACHTUNG: $s_k @ y_k = -0.0037444770938565575$
 ACHTUNG: $s_k @ y_k = -0.20506869913926162$
 ACHTUNG: $s_k @ y_k = -3.8006798236219193$
 ACHTUNG: $s_k @ y_k = -0.002051756974308547$
 ACHTUNG: $s_k @ y_k = -0.003949166295357764$
 ACHTUNG: $s_k @ y_k = -0.0003154215532175761$
 ACHTUNG: $s_k @ y_k = -2.759574310899908e-06$
 ACHTUNG: $s_k @ y_k = -9.787894801538616e-10$
 ACHTUNG: $s_k @ y_k = -1.7118179731820838e-05$

ACHTUNG: sk @ yk = -8.75061709021852e-06
ACHTUNG: sk @ yk = -2.6617468389952406e-08
ACHTUNG: sk @ yk = -6.096962658587201e-09
ACHTUNG: sk @ yk = -4.031479708587146e-06
ACHTUNG: sk @ yk = -6.959061270583825e-07
ACHTUNG: sk @ yk = -4.100539391038662e-07
Abbruch nach GMW 1 erfuehlt
Abbruch nach GMW 1 erfuehlt
Abbruch nach GMW 1 erfuehlt
Abbruch nach GMW 3 erfuehlt
ACHTUNG: sk @ yk = -11.771513422694523
ACHTUNG: sk @ yk = -0.9493748375564728
ACHTUNG: sk @ yk = -1.8338411444528926e-16
ACHTUNG: sk @ yk = -0.0051702675027561346
ACHTUNG: sk @ yk = -5.582001897828405e-05
ACHTUNG: sk @ yk = -0.43259355204871497
ACHTUNG: sk @ yk = 0.0

Liniensuche wurde nach 100 Schritten fruehzeitig abgebrochen.
Liniensuche wurde nach 100 Schritten fruehzeitig abgebrochen.
Liniensuche wurde nach 100 Schritten fruehzeitig abgebrochen.
Liniensuche wurde nach 100 Schritten fruehzeitig abgebrochen.
Liniensuche wurde nach 100 Schritten fruehzeitig abgebrochen.
Liniensuche wurde nach 100 Schritten fruehzeitig abgebrochen.
Liniensuche wurde nach 100 Schritten fruehzeitig abgebrochen.
Liniensuche wurde nach 100 Schritten fruehzeitig abgebrochen.
Liniensuche wurde nach 100 Schritten fruehzeitig abgebrochen.
Liniensuche wurde nach 100 Schritten fruehzeitig abgebrochen.
Liniensuche wurde nach 100 Schritten fruehzeitig abgebrochen.
Liniensuche wurde nach 100 Schritten fruehzeitig abgebrochen.
Liniensuche wurde nach 100 Schritten fruehzeitig abgebrochen.
Liniensuche wurde nach 100 Schritten fruehzeitig abgebrochen.
Liniensuche wurde nach 100 Schritten fruehzeitig abgebrochen.
Liniensuche wurde nach 100 Schritten fruehzeitig abgebrochen.
Liniensuche wurde nach 100 Schritten fruehzeitig abgebrochen.
Liniensuche wurde nach 100 Schritten fruehzeitig abgebrochen.
Liniensuche wurde nach 100 Schritten fruehzeitig abgebrochen.
Liniensuche wurde nach 100 Schritten fruehzeitig abgebrochen.
Liniensuche wurde nach 100 Schritten fruehzeitig abgebrochen.

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

Linienuche wurde nach 100 Schritten frühzeitig abgebrochen.

Linienuche wurde nach 100 Schritten frühzeitig abgebrochen.

Linienuche wurde nach 100 Schritten frühzeitig abgebrochen.

Abbruch nach GMW 3 erfuehlt

ACHTUNG: sk @ yk = -7.793650155063592

ACHTUNG: sk @ yk = 2.465190328815662e-30

ACHTUNG: sk @ yk = -0.004267457158845134

ACHTUNG: sk @ yk = 6.162975822039155e-31

ACHTUNG: sk @ yk = 6.162975822039155e-31

ACHTUNG: sk @ yk = 9.860761315262648e-31

ACHTUNG: sk @ yk = 6.162975822039155e-31

ACHTUNG: sk @ yk = 1.7256332301709633e-31

ACHTUNG: sk @ yk = 7.888609052210118e-31

ACHTUNG: sk @ yk = 1.67632942359465e-30

ACHTUNG: sk @ yk = 8.874685183736383e-31

ACHTUNG: sk @ yk = 6.162975822039155e-31

ACHTUNG: sk @ yk = 8.874685183736383e-31

ACHTUNG: sk @ yk = 1.67632942359465e-30

ACHTUNG: sk @ yk = 7.888609052210118e-31

ACHTUNG: sk @ yk = 1.67632942359465e-30

ACHTUNG: sk @ yk = 9.860761315262648e-31

ACHTUNG: sk @ yk = 6.162975822039155e-31

ACHTUNG: sk @ yk = 9.860761315262648e-31

ACHTUNG: sk @ yk = 6.162975822039155e-31

ACHTUNG: sk @ yk = 6.162975822039155e-31

ACHTUNG: sk @ yk = 6.162975822039155e-31

ACHTUNG: sk @ yk = 6.162975822039155e-31

ACHTUNG: sk @ yk = 6.162975822039155e-31

ACHTUNG: sk @ yk = 6.162975822039155e-31

ACHTUNG: sk @ yk = 6.162975822039155e-31

ACHTUNG: sk @ yk = 6.162975822039155e-31

ACHTUNG: sk @ yk = 6.162975822039155e-31

ACHTUNG: sk @ yk = 6.162975822039155e-31

ACHTUNG: sk @ yk = 6.162975822039155e-31

ACHTUNG: sk @ yk = 6.162975822039155e-31

ACHTUNG: sk @ yk = 6.162975822039155e-31

ACHTUNG: sk @ yk = 6.162975822039155e-31

ACHTUNG: sk @ yk = 6.162975822039155e-31

ACHTUNG: sk @ yk = 6.162975822039155e-31
ACHTUNG: sk @ yk = 6.162975822039155e-31
ACHTUNG: sk @ yk = 6.162975822039155e-31
ACHTUNG: sk @ yk = 6.162975822039155e-31
ACHTUNG: sk @ yk = 6.162975822039155e-31
ACHTUNG: sk @ yk = 6.162975822039155e-31
ACHTUNG: sk @ yk = 6.162975822039155e-31
ACHTUNG: sk @ yk = 6.162975822039155e-31
ACHTUNG: sk @ yk = 6.162975822039155e-31
ACHTUNG: sk @ yk = 7.888609052210118e-31
ACHTUNG: sk @ yk = 6.162975822039155e-31
ACHTUNG: sk @ yk = 6.162975822039155e-31
ACHTUNG: sk @ yk = 6.162975822039155e-31
ACHTUNG: sk @ yk = 6.162975822039155e-31
ACHTUNG: sk @ yk = 6.162975822039155e-31
ACHTUNG: sk @ yk = 6.162975822039155e-31
ACHTUNG: sk @ yk = 6.162975822039155e-31
ACHTUNG: sk @ yk = 6.162975822039155e-31
ACHTUNG: sk @ yk = 6.162975822039155e-31
ACHTUNG: sk @ yk = 6.162975822039155e-31
ACHTUNG: sk @ yk = 6.162975822039155e-31
ACHTUNG: sk @ yk = 7.888609052210118e-31
ACHTUNG: sk @ yk = 1.873544649899903e-30
ACHTUNG: sk @ yk = 6.162975822039155e-31
ACHTUNG: sk @ yk = 7.888609052210118e-31
ACHTUNG: sk @ yk = 1.67632942359465e-30
ACHTUNG: sk @ yk = 7.888609052210118e-31
ACHTUNG: sk @ yk = 1.7749370367472766e-30
ACHTUNG: sk @ yk = 8.874685183736383e-31
ACHTUNG: sk @ yk = 7.888609052210118e-31
ACHTUNG: sk @ yk = 1.67632942359465e-30
ACHTUNG: sk @ yk = 7.888609052210118e-31
ACHTUNG: sk @ yk = 1.873544649899903e-30
ACHTUNG: sk @ yk = 7.888609052210118e-31
ACHTUNG: sk @ yk = 7.888609052210118e-31
ACHTUNG: sk @ yk = 1.67632942359465e-30
ACHTUNG: sk @ yk = 7.888609052210118e-31
ACHTUNG: sk @ yk = 1.873544649899903e-30
ACHTUNG: sk @ yk = 7.888609052210118e-31

ACHTUNG: sk @ yk = 1.67632942359465e-30
 ACHTUNG: sk @ yk = 7.888609052210118e-31
 ACHTUNG: sk @ yk = 8.874685183736383e-31
 ACHTUNG: sk @ yk = 1.7749370367472766e-30
 ACHTUNG: sk @ yk = 7.888609052210118e-31
 ACHTUNG: sk @ yk = 1.67632942359465e-30
 ACHTUNG: sk @ yk = 7.888609052210118e-31
 ACHTUNG: sk @ yk = 1.873544649899903e-30
 ACHTUNG: sk @ yk = 7.888609052210118e-31
 ACHTUNG: sk @ yk = 7.888609052210118e-31
 ACHTUNG: sk @ yk = 1.67632942359465e-30
 ACHTUNG: sk @ yk = 7.888609052210118e-31
 ACHTUNG: sk @ yk = 1.7749370367472766e-30
 ACHTUNG: sk @ yk = 8.874685183736383e-31
 ACHTUNG: sk @ yk = 1.67632942359465e-30
 ACHTUNG: sk @ yk = 7.888609052210118e-31
 ACHTUNG: sk @ yk = 8.874685183736383e-31
 ACHTUNG: sk @ yk = 1.7749370367472766e-30
 ACHTUNG: sk @ yk = 7.888609052210118e-31
 ACHTUNG: sk @ yk = 1.67632942359465e-30
 ACHTUNG: sk @ yk = 7.888609052210118e-31
 ACHTUNG: sk @ yk = 9.860761315262648e-31
 ACHTUNG: sk @ yk = 1.5777218104420236e-30
 ACHTUNG: sk @ yk = 6.162975822039155e-31
 ACHTUNG: sk @ yk = 6.162975822039155e-31
 ACHTUNG: sk @ yk = 6.162975822039155e-31
 ACHTUNG: sk @ yk = 6.162975822039155e-31
 ACHTUNG: sk @ yk = 6.162975822039155e-31
 ACHTUNG: sk @ yk = 6.162975822039155e-31
 ACHTUNG: sk @ yk = 6.162975822039155e-31
 ACHTUNG: sk @ yk = 6.162975822039155e-31
 ACHTUNG: sk @ yk = 6.162975822039155e-31
 ACHTUNG: sk @ yk = 2.1200636827814692e-30
 ACHTUNG: sk @ yk = 5.423418723394456e-31
 ACHTUNG: sk @ yk = 7.888609052210118e-31
 ACHTUNG: sk @ yk = 7.888609052210118e-31
 ACHTUNG: sk @ yk = 3.0814879110195774e-30
 ACHTUNG: sk @ yk = 6.162975822039155e-31

Abbruch nach GMW 1 erfuehlt

Abbruch nach GMW 1 erfuehlt

Abbruch nach GMW 3 erfuehlt

Es zeigt sich, dass Newton die wenigsten Iterationen benoetigt, dies geht natuerlich auf Kosten von Speicherbedarf und Rechenzeit.

Das Gradientenverfahren benoetigt immer die kmax Iteration und terminiert durch GMW 3. Der berechnete Wert ist daher nicht optimal.

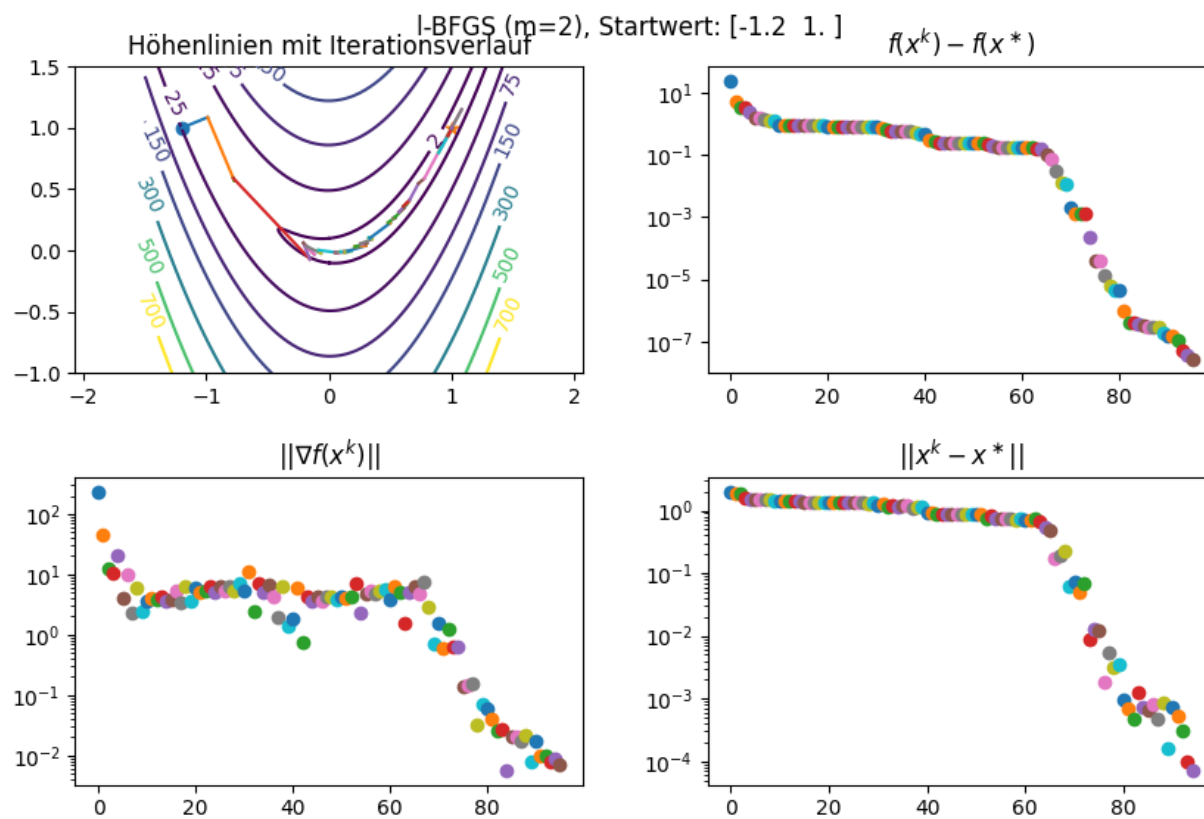
Das BFGS-Verfahren und das l-BFGS-Verfahren benoetigen beide (deutlich) weniger Iterationen als das Gradientenverfahren,

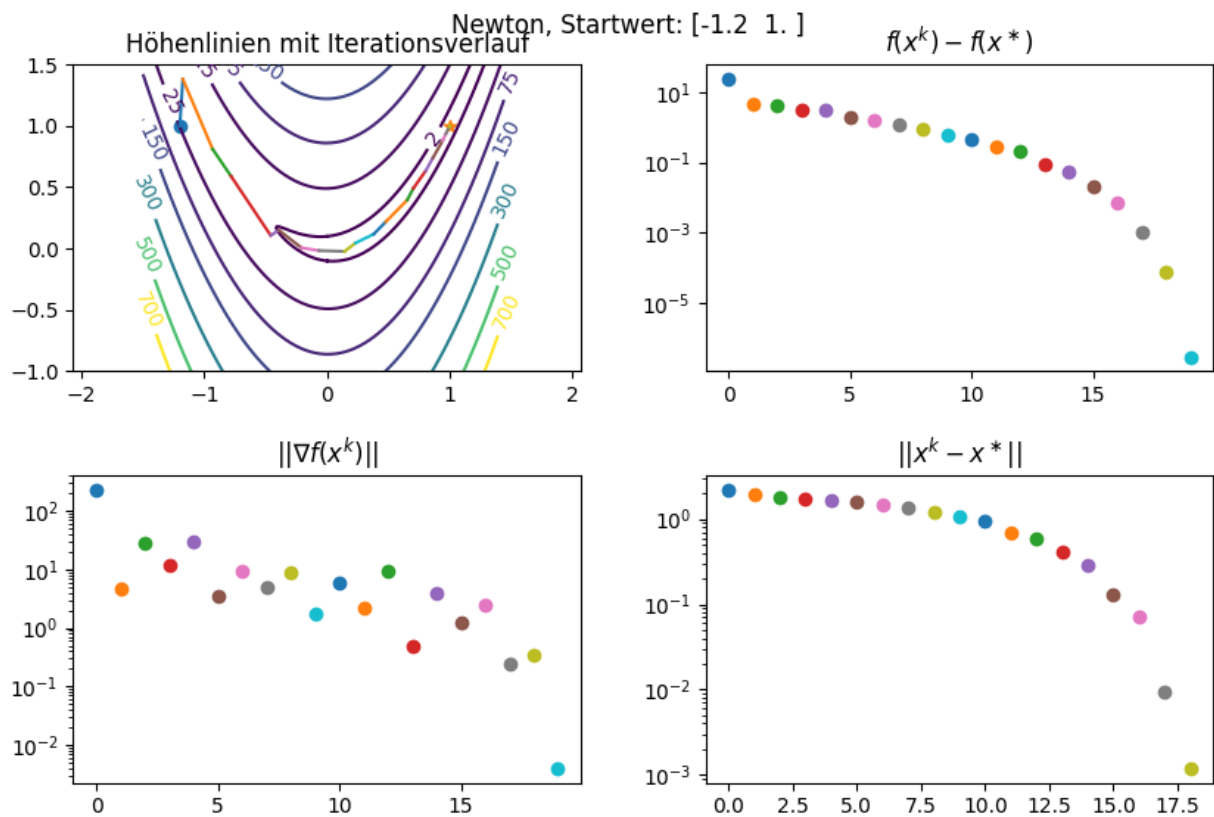
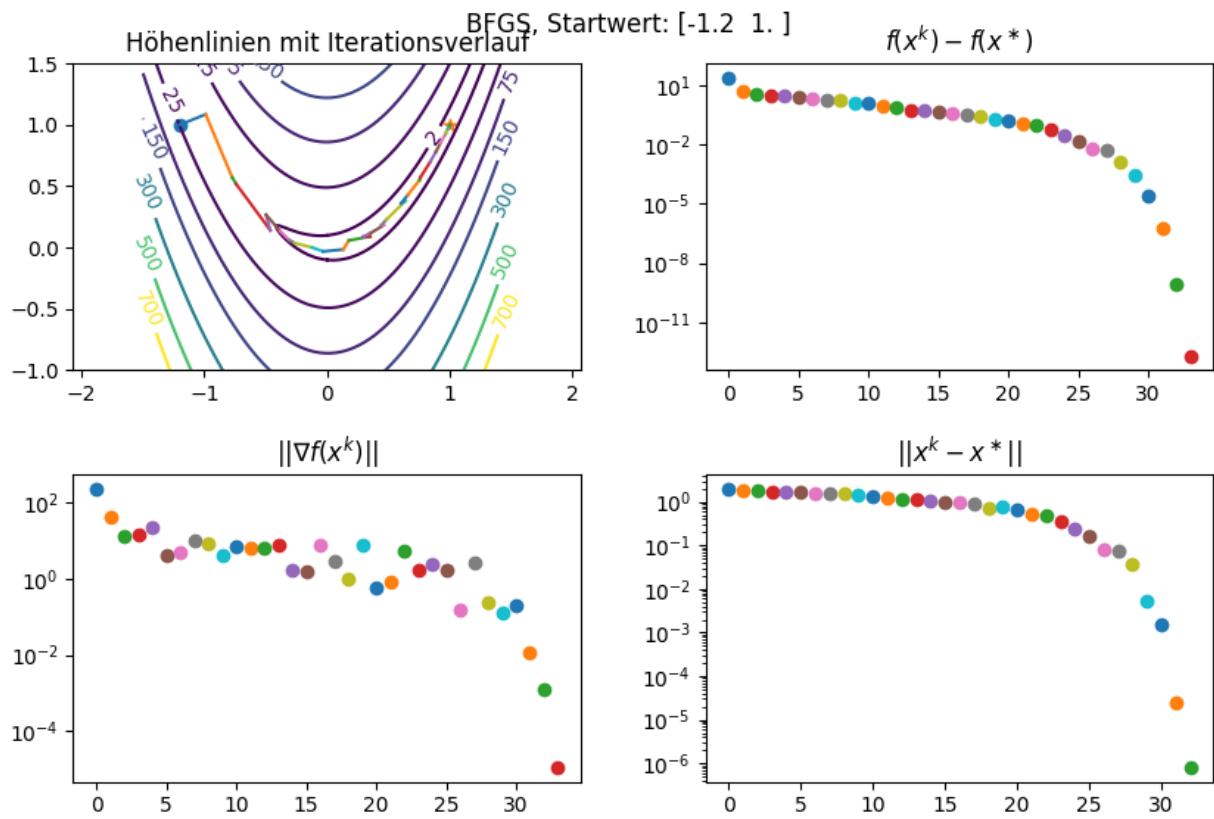
wobei l-BFGS deutlich weniger Speicher benoetigt. Dabei ist zu beobachten, dass abhaengig vom Startwert jeweils eins der beiden

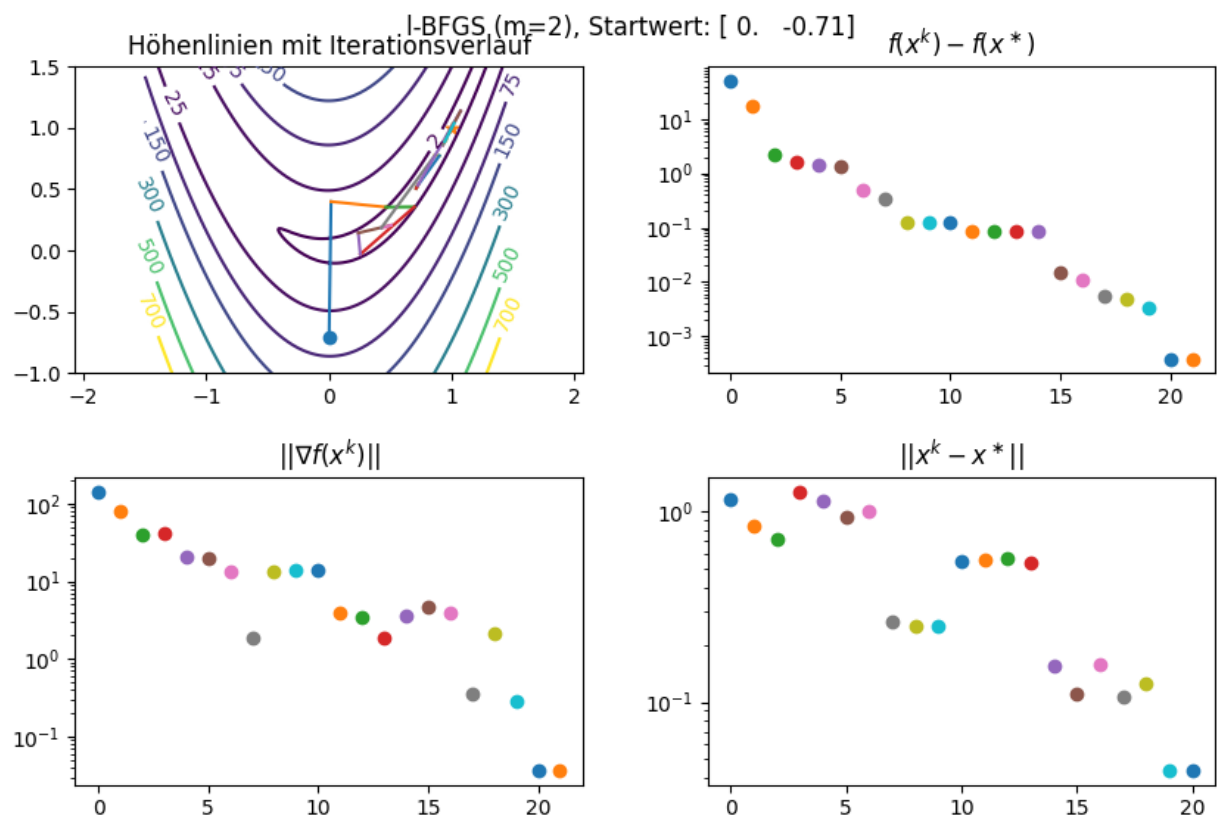
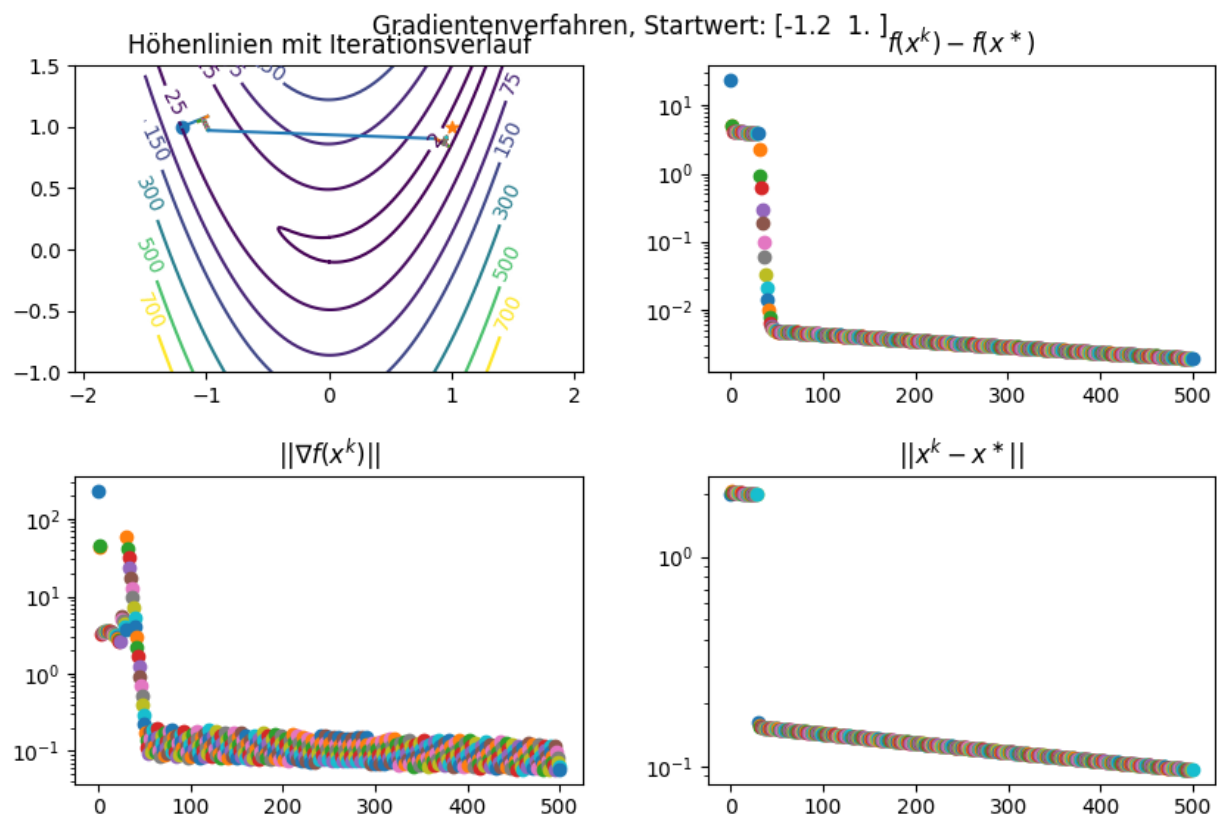
Verfahren fast so schnell wie Newton (nach Iterationen) konvergiert, beim anderen Punkt aber deutlich schlechter.

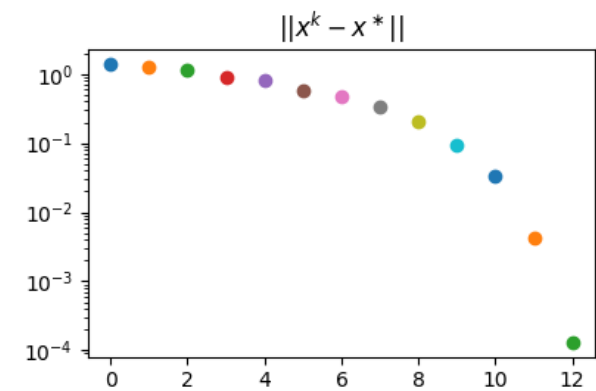
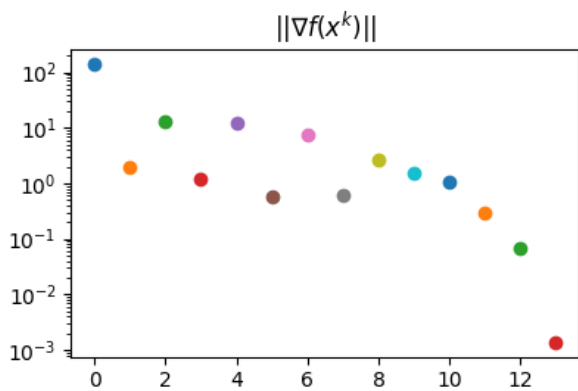
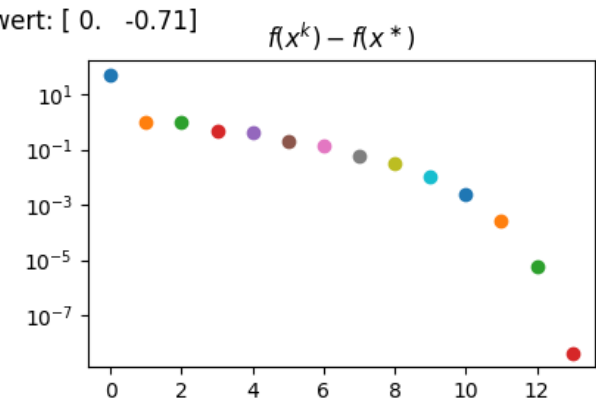
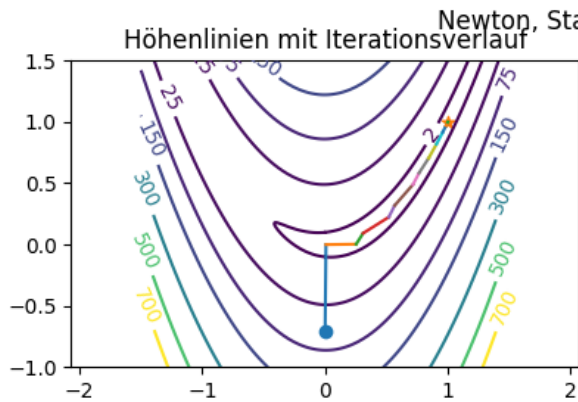
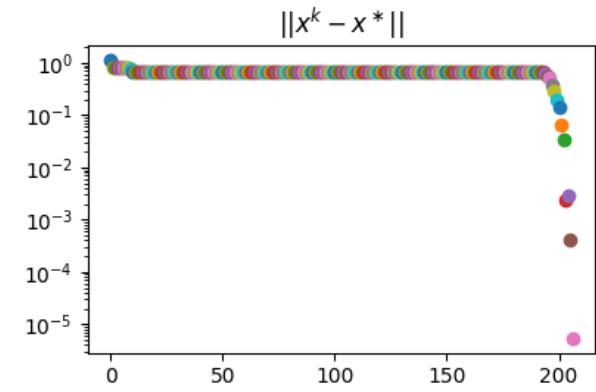
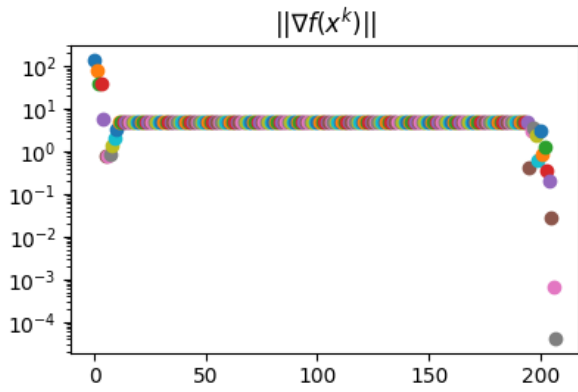
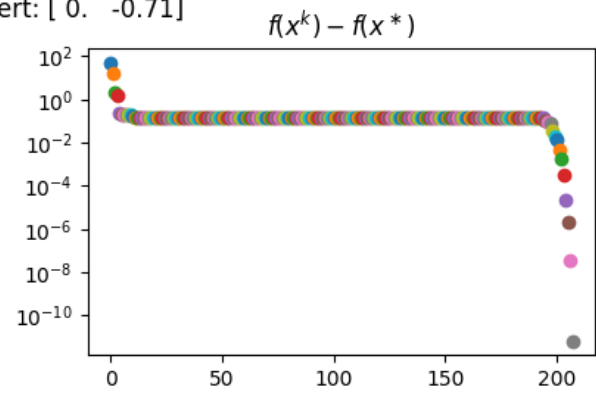
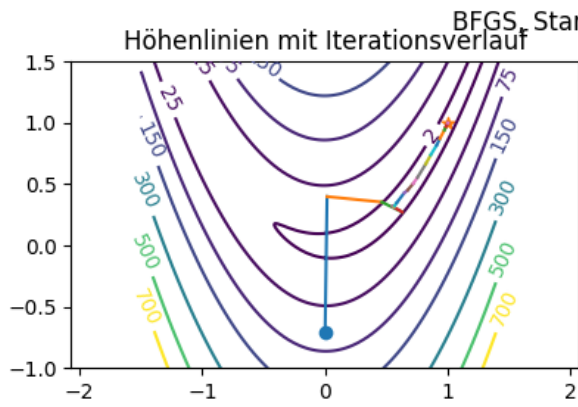
Wobei selbst hier das l-BFGS-Verfahren nicht so viele Iterationen benoetigt wie das BFGS beim anderen Ausgangspunkt.

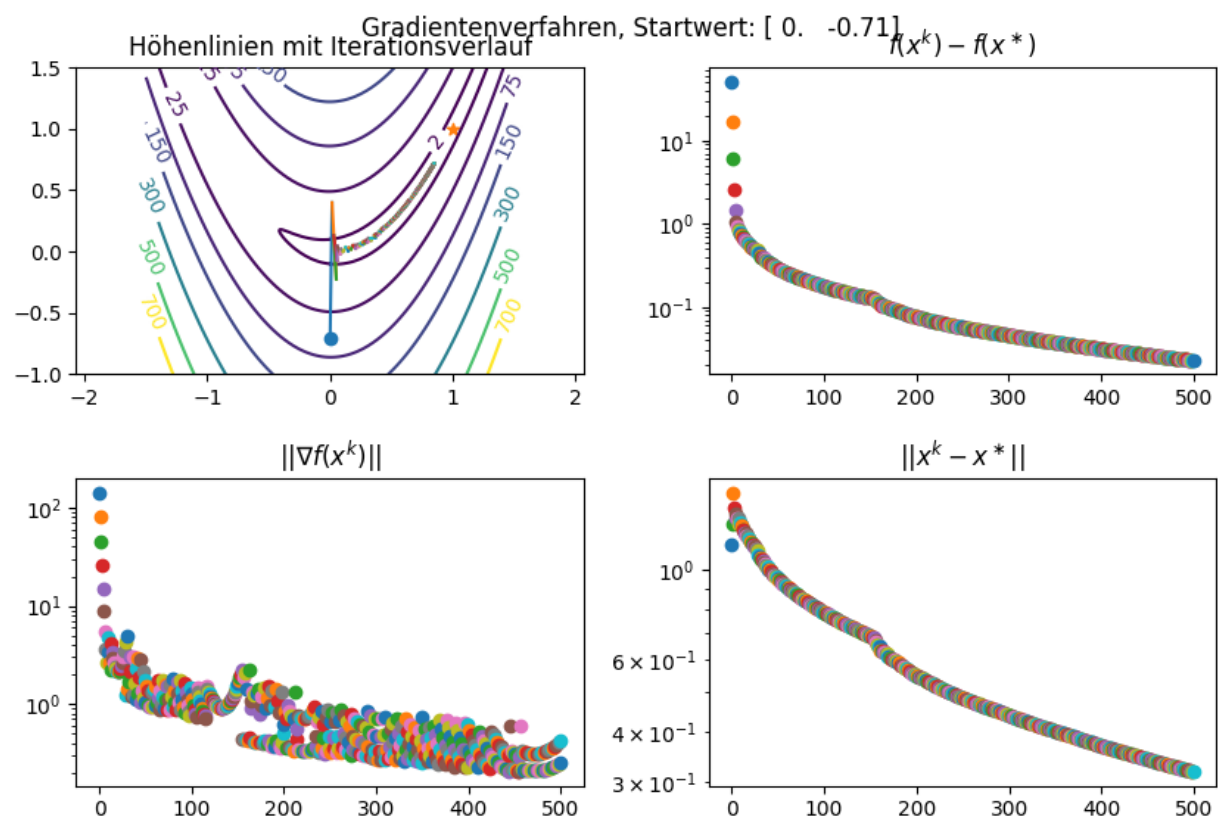
#PLOTS:











```

import matplotlib.pyplot as plt
import numpy as np

# Uebernommen von Blatt 8
def armijo(f, p, x, rho, c1, alpha0=1):
    """
    Armijo Backtracking Liniensuche

    Input:
        f: Funktion, wobei  $f(x) = (val, grad, hess)$ ,
        p: Vektor (Abstiegsrichtung)
        x: Vektor (Iterierte)
        rho: Skalar (Reduktionsfaktor für alpha)
        c1: Skalar (Parameter für die Armijo-Bedingung)
        alpha0: Skalar (Initial getesteter Startwert)

    Output:
        alpha: Skalar (gesuchte Schrittweite)
    """

    # Initialisierung
    alpha = alpha0

    # Vorberechnung reduziert Laufzeit in der Schleife
    valk, gradk, _ = f(x)
    descent = gradk @ p

    for lsiter in range(100):
        valkp1, _, _ = f(x + alpha * p)

        # Testen der ``sufficient decrease condition``
        if valkp1 <= valk + c1 * alpha * descent:
            return alpha

        # Reduzieren von alpha
        alpha = rho * alpha

    # Fehlermeldung, falls die Liniensuche nicht auskonvergiert ist
    print("Liniensuche wurde nach 100 Schritten fröhzeitig abgebrochen.")
    return alpha

# Uebernommen von Blatt 8
def gill_murray_wright(xk, xkplus1, valk, valkplus1, gradk, k, eps, tau, kmax):
    """
    Abbruchkriterien nach Gill, Murray und Wright

    Input:
        xk: Vektor (aktuelle Iterierte)
        xkp1: Vektor (nächste Iterierte)
        valk: Skalar ( $f(x_k)$ )
        valkp1: Skalar ( $f(x_{k+1})$ )
    """

```

```

gradk: Vektor ( $\nabla f(x_k)$ )
k: Integer (Iteration-Nummer)
tau: Skalar (Toleranz für Kriterium 1)
eps: Skalar (Toleranz für Gradient -- Kriterium 3)
kmax: Integer (maximale Anzahl der Iterationen)

```

Output:

```

test: Boolescher Wert (Ergebnis der Abbruchkriterien nach Gill, Murray, Wright)
"""

```

```

test = False
if (valk - valkplus1 < tau * (1 + np.abs(valk))
    and np.linalg.norm(xkplus1 - xk) < np.sqrt(tau) * (1 + np.linalg.norm(xk))
    and np.linalg.norm(gradk) < np.cbrt(tau) * (1 + np.abs(valk))):
    test = True
    print('\nAbbruch nach GMW 1 erfuehlt')

if np.linalg.norm(gradk) < eps:
    test = True
    print('\nAbbruch nach GMW 3 erfuehlt')

if k == kmax:
    test = True
    print('\nAbbruch nach GMW 3 erfuehlt')

return test

```

Uebernomen von Blatt 8

```

def gradient_descent_erweitert(f, x0, rho=0.5, c1=1e-3, eps=1e-10, tau=1e-10, kmax=100, xstar=None):
    """

```

Gradientenabstiegsverfahren

Input:

```

f: Funktion, wobei  $f(x) = (val, grad, hess)$ ,
x0: Vektor (Startwert),
rho: Skalar (Armijo Parameter für die Anpassung der Schrittweite)
c1: Skalar (Armijo Parameter für das Testen der Bedingung),
tau: Skalar (Toleranz für Kriterium 1),
eps: Skalar (Toleranz für Gradient in GMW-Kriterium 3),
kmax: Skalar (Maximale Anzahl an Iterationen),
xstar: Vector (Minimierer von f), optional

```

Output:

```

log: Dictionary vom Verlauf der Optimierung.
"""

```

Dictionary zum Abspeichern der Ergebnisse.

```

log = {
    'xstar': xstar,          # Tatsächlicher Minimierer
    'val_star': None if xstar is None else f(xstar)[0], # Tatsächliches Minimum
    'x0': x0,               # Startwert
    'xgd': None,            # Lösung des Gradientenabstiegsverfahrens
    'kgd': None,            # Anzahl benötigter Iterationen
    'x_list': [],           # Liste der Iterierten

```



```

        'val_list': [],          # Liste des Funktionswerts in den Iterierten
        'norm_grad_list': [],   # Liste der Norm des Gradienten in den Iterierten
    }

    k = 0
    xk = x0
    while True:
        valk, gradk, hessk = f(xk)

        log['x_list'].append(xk)
        log['val_list'].append(valk)
        log['norm_grad_list'].append(np.linalg.norm(gradk))

        pk = -gradk
        alpha0 = max(- pk @ gradk / (pk @ hessk @ pk), 1)
        alphak = armijo(f, pk, xk, rho=rho, c1=c1, alpha0=alpha0) # ---- NEU: Armijo Liniensuche ----
        xkplus1 = xk + alphak * pk

        if gill_murray_wright(xk, xkplus1, valk, valkplus1=f(xkplus1)[0], gradk=gradk, k=k, tau=tau, eps=eps,
                               kmax=kmax):
            break

        k += 1
        xk = xkplus1

    log['xgd'] = xk
    log['kgd'] = k
    return log

```

Uebernommen von Blatt 8

```

def newton_erweitert(f, x0, rho=0.5, c1=1e-3, eps=1e-10, tau=1e-10, kmax=100, xstar=None):
    """

```

Newton-Verfahren zur Minimierung der Funktion f.

Input:

*f: Funktion, wobei $f(x) = (val, grad, hess)$,
 x0: Vektor (Startwert),
 rho: Skalar (Armijo Parameter für die Anpassung der Schrittweite)
 c1: Skalar (Armijo Parameter für das Testen der Bedingung),
 tau: Skalar (Toleranz für Kriterium 1),
 eps: Skalar (Toleranz für Gradient in GMW-Kriterium 3),
 kmax: Skalar (Maximale Anzahl an Iterationen),
 xstar: Vector (Minimierer von f), optional*

Output:

log: Dictionary vom Verlauf der Optimierung.
 """

Dictionary zum Abspeichern der Ergebnisse.

```

log = {
    'xstar': xstar,          # Tatsächlicher Minimierer
    'val_star': None if xstar is None else f(xstar)[0], # Tatsächliches Minimum
    'x0': x0,               # Startwert
    'xnv': None,            # Lösung des Newton-Verfahrens

```

```

        'knv': None,                # Anzahl benötigter Iterationen
        'x_list': [],              # Liste der Iterierten
        'val_list': [],            # Liste des Funktionswerts in den Iterierten
        'norm_grad_list': [],      # Liste der Norm des Gradienten in den Iterierten
    }

    # Initialisierung
    k = 0
    xk = x0

    # Iteration
    while True:
        valk, gradk, hessk = f(xk)
        log['x_list'].append(xk)
        log['val_list'].append(valk)
        log['norm_grad_list'].append(np.linalg.norm(gradk))

        # Newton Schritt
        pk = np.linalg.solve(hessk, -gradk)
        alphak = armijo(f, pk, xk, rho=rho, c1=c1) # ---- NEU: Armijo Liniensuche ----
        xkplus1 = xk + alphak * pk

        if gill_murray_wright(xk, xkplus1, valk, valkplus1=f(xkplus1)[0], gradk=gradk, k=k, tau=tau, eps=
                               kmax=kmax):
            break

        # Update
        k += 1
        xk = xkplus1

    log['xnv'] = xk
    log['knv'] = k
    return log

# Uebernommen von Blatt 9
def quasi_newton(f, x0, B0, rho=0.5, c1=1e-3, eps=1e-10, tau=1e-10, kmax=100, plot=True, display=True):
    """
    BFGS-basiertes Quasi-Newton-Verfahren zur Minimierung der Funktion f.

    Input:
        f: Funktion, wobei f(x) = (val, grad, hess),
        x0: Vektor (Startwert),
        B0: Matrix (Initiale Schätzung für die inverse Hesse-Matrix),
        rho, c1: Skalare (Armijo Parameter),
        tau, eps, kmax: Skalare (Abbruchkriterien nach Gill-Murray-Wright),
        plot: Boolescher Wert (gibt an, ob Daten zur Ploterstellung gespeichert werden sollen),
        display: Boolescher Wert (gibt an, ob Zwischenwerte aus den Iterationen in der Konsole ausgegeben

    Output:
        log: Dictionary vom Verlauf der Optimierung.
    """

    # Dictionary zum Abspeichern der Ergebnisse.
    log = {
        'x0': x0,                # Startwert

```

```

'xqn': None,          # Lösung des CG-Verfahrens
'kqn': None,          # Anzahl benötigter Iterationen
'x_list': [],         # Liste der Iterierten
'val_list': [],       # Liste des Funktionswerts in den Iterierten
'norm_grad_list': [], # Liste der Norm des Gradienten in den Iterierten
}

# Initialisierung
xk = x0
valk, gradk, _ = f(xk)
Bk = B0
k = 0
I = np.eye(xk.size)

# Iteration
while True:
    if plot:
        log['x_list'].append(xk)
        log['val_list'].append(valk)
        log['norm_grad_list'].append(np.linalg.norm(gradk))

    # Iterationsschritt
    pk = - Bk @ gradk
    alphak = armijo(f, p=pk, x=xk, rho=rho, c1=c1)

    if display:
        print(f'Iteration k = {k}: \talpha_k = {alphak: .4e}, '
              f'\txk = {str(np.round(xk, 4).flatten()): <30}, \tf(xk) = {np.round(valk, 4)}')

    xkplus1 = xk + alphak * pk
    valkplus1, gradkplus1, _ = f(xkplus1)

    # Test der Abbruchkriterien
    if gill_murray_wright(xk, xkplus1, valk, valkplus1, gradk, k, tau, eps, kmax):
        break

    # Anpassung von Bk
    sk = xkplus1 - xk
    yk = gradkplus1 - gradk

    rhok = 1 / (yk @ sk)
    Bkplus1 = (I - rhok * np.outer(sk, yk)) @ Bk @ (I - rhok * np.outer(yk, sk)) + rhok * np.outer(sk,
    yk)

    # Warnung, wenn yk @ sk nicht > 0
    if sk @ yk < 1e-12:
        print(f"ACHTUNG: sk @ yk = {sk @ yk}")

    # Update
    xk, valk, gradk = xkplus1, valkplus1, gradkplus1
    Bk = Bkplus1
    k += 1

log['xqn'] = xk
log['kqn'] = k
return log

```

```

# Uebernomen von Blatt 8
def rosenbrock(x, y):
    """
    Implementierung der Rosenbrock-Funktion  $f(x, y) = 100(y-x^2)^2 + (1-x)^2$ 

    Input:
        x, y: Skalare bzw. Arrays zur Auswertung der Funktion.

    Output:
        val: Zielfunktionswert(e),
        grad: Gradient(en),
        hess: Hessematri(x/zen).
    """
    val = 100 * (y - x ** 2) ** 2 + (1 - x) ** 2
    grad = np.stack((- 400 * x * (y - x ** 2) - 2 * (1 - x), 200 * (y - x ** 2)), axis=-1)
    hess = np.stack((np.stack((-400 * (y - x ** 2) + 800 * x ** 2 + 2, -400 * x), axis=-1),
                     np.stack((-400 * x, 200 * np.ones_like(x)), axis=-1)), axis=1)

    return val, grad, hess

# Angepasst von Blatt 8
def plot_iteration_rosenbrock(log, title):
    """
    Funktion zur Erstellung von Plots zum Optimierungsverlauf der Rosenbrock-Funktion anhand der Ergebnisse
    Im ersten Plot wird der Iterationsverlauf auf einem Surface-Plot dargestellt, in weiteren 3 Konvergenz
    Die Funktion liefert fig zur  ck. Stellen Sie diese dar, indem Sie plt.show( ) in Ihrer Hauptdatei au  f

    Input:
        log: Dictionary, das mindestens folgende Eintr  ge enth  lt:
            log = {
                'x0': Vektor,                # Startwert
                'x_list': Liste,              # Iterierte
                'val_list': Liste,            # Funktion ausgewertet an den Iterierten
                'norm_grad_list': Liste,      # Norm des Gradienten ausgewertet an dem Iterierten
            }
        title: Titel des Plots.

    Output:
        fig: hergestellte Figure mit den Plots
    """

    # Figure initialisieren
    fig, ax = plt.subplots(2, 2)
    fig.set_size_inches(9, 6)
    fig.tight_layout(pad=3, h_pad=3)
    fig.suptitle(title)
    epsilon = np.finfo(float).eps # Vermeide Probleme mit Teilen durch 0 in den Plots sp  ter

    # Surface-Plot mit Iterationsverlauf
    x = np.linspace(-1.5, 1.5, 651)
    y = np.linspace(-1., 1.5, 651)
    xx, yy = np.meshgrid(x, y)
    zz = rosenbrock(xx.flatten(), yy.flatten())[0].reshape((651, 651))

```

```

CS = ax[0, 0].contour(xx, yy, zz, levels=[2, 25, 75, 150, 300, 500, 700])
ax[0, 0].clabel(CS, inline=1, fontsize=10)
ax[0, 0].scatter(*log['x0'], marker='o')
ax[0, 0].scatter(1, 1, marker='*')
for k in range(len(log['x_list']) - 1):
    xk = log['x_list'][k]
    xkplus1 = log['x_list'][k + 1]
    ax[0, 0].plot([xk[0], xkplus1[0]], [xk[1], xkplus1[1]])
ax[0, 0].set_title('Hhenlinien mit Iterationsverlauf')
ax[0, 0].set_aspect('equal', 'datalim') # Um die Orthogonalitt der Suchrichtungen besser zu visuali

# Plotten der Zielfunktionsfehler
val_list = log['val_list']
val_star = 0
for k, valk in enumerate(val_list):
    ax[0, 1].scatter(k, valk - val_star)
ax[0, 1].set_yscale('log')
ax[0, 1].set_title(r'$f(x^k) - f(x^ast)$')

# Plotten der Gradientennorm
norm_grad_list = log['norm_grad_list']
for k, norm_gradk in enumerate(norm_grad_list):
    ax[1, 0].scatter(k, norm_gradk)
ax[1, 0].set_yscale('log')
ax[1, 0].set_title(r'$||\nabla f(x^k) ||$')

# Plotten der Konvergenz der Iterierten
x_list = log['x_list']
xstar = np.array([1, 1])
for k, xk in enumerate(x_list[1:]):
    ax[1, 1].scatter(k, np.linalg.norm(xk - xstar))
ax[1, 1].set_yscale('log')
ax[1, 1].set_title(r'$||x^k - x^ast||$')

return fig

```

#TERMINAL OUTPUT:

#PLOTS: